

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Angelo Morzenti

Exam of MONDAY 2 SEPTEMBER 2024 – Theory Part

WITH SOLUTIONS – FOR TEACHING PURPOSES THE SOLUTIONS ARE WIDELY COMMENTED

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. A. MORZENTI

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): syntax and semantics of languages, divided in four sections:
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): compiler design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- For the theory part to be valid, the candidate must achieve a grade of at least 10/20 on each of the four sections.
- Correctly answering all the questions that are not marked optional, allows the candidate to achieve a high grade (although not the full mark).
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30.
- The final grade is the weighted average of the theory part (80 %) and of the lab part (20 %), and must be $\geq 18/30$.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

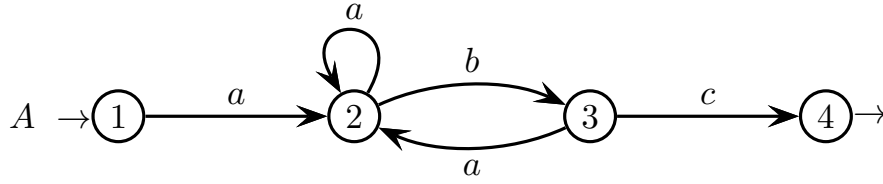
1 Regular Expressions and Finite Automata 20%

1. Consider the two regular expressions R_1 and R_2 below, over the four-letter alphabet $\{a, b, c, d\}$:

$$R_1 = ((a^+ b)^+ c)^+ d$$

$$R_2 = a^* b^* c^* d^*$$

and the automaton A below, over the three-letter alphabet $\{a, b, c\}$:



Answer the following questions:

- (a) Consider the regular expression R_1 above. Write one string of length 5 and two strings of length 6 generated by R_1 .
- (b) Write a derivation from the regular expression R_1 above, for the sample valid string below (of length 7):

$a a b a b c d$

- (c) Apply the Berry-Sethi (BS) method to the regular expression R_1 above, to obtain an equivalent deterministic automaton A_1 .
- (d) Say if the automaton A_1 obtained through the BS method at point (c) is minimal, and explain your answer. Minimize automaton A_1 if it is not minimal.
- (e) Apply the Brzozowski-McCluskey (BMC – node elimination) method to the automaton A above (**not to** A_1), to obtain an equivalent regular expression R .
- (f) (optional) Consider the regular expressions R_1 and R_2 above and their intersection language $L_\cap$, below:

$$L_\cap = L(R_1) \cap L(R_2) = L(((a^+ b)^+ c)^+ d) \cap L(a^* b^* c^* d^*)$$

Write a regular expression $R_\cap$ that generates language $L_\cap$, without using the intersection operator “ $\cap$ ” of sets. Briefly explain your answer.

Solution

- (a) Here are the only string of length 5 and the two only strings of length 6 generated by the regular expression R_1 :

$a a b c d$ $a a a b c d$ $a b a b c d$

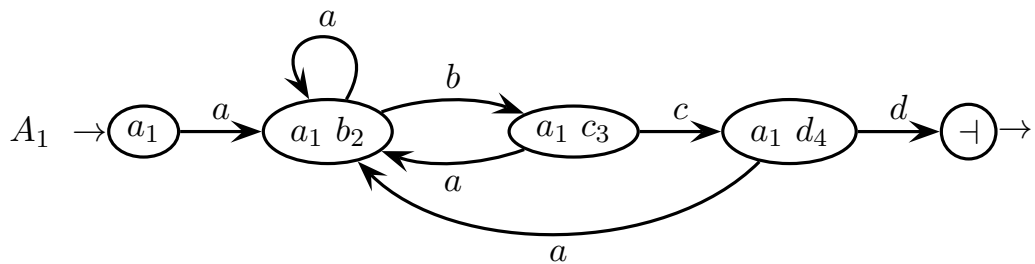
- (b) Here is the derivation of the valid string $a a b a b c d$ (of length 7) from the regular expression R_1 , where the choices for the cross operator are coloured in **red**:

$$\begin{aligned}
 R_1 &= ((a^+ b)^+ c)^{\overset{1}{+}} d \\
 &\Rightarrow (a^+ b)^{\overset{2}{+}} c d \\
 &\Rightarrow (a^{\overset{2}{+}} b) (a^+ b) c d \\
 &\Rightarrow a a b (a^{\overset{1}{+}} b) c d \\
 &\Rightarrow a a b a b c d
 \end{aligned}$$

- (c) Here is the application of the Berry-Sethi (BS) method to the regular expression R_1 , to obtain an equivalent deterministic automaton A_1 (with five states):

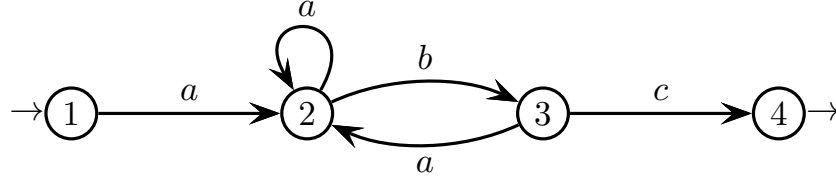
$$R_{1\#} = ((a_1^+ b_2)^+ c_3)^+ d_4 \dashv$$

initials	a_1
terminal	followers
a_1	$a_1 b_2$
b_2	$a_1 c_3$
c_3	$a_1 d_4$
d_4	$\dashv$

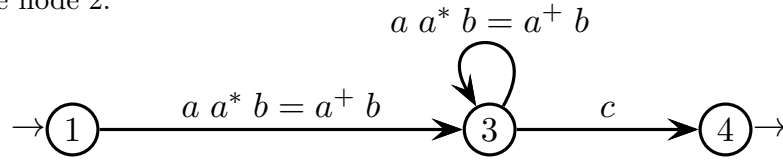


- (d) The deterministic automaton A_1 is minimal. Only one state is final and therefore is distinguishable from all the other non-final states. The four non-final states have different labels on their outgoing arcs and therefore are distinguishable.

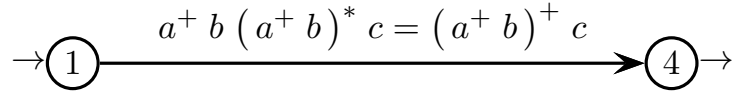
- (e) Here is the application of the Brzozowski-McCluskey (BMC) method to automaton A , to obtain an equivalent regular expression R . It is not necessary to normalize the input node 1 and the output node 4 of the starting automaton:



Eliminate node 2:



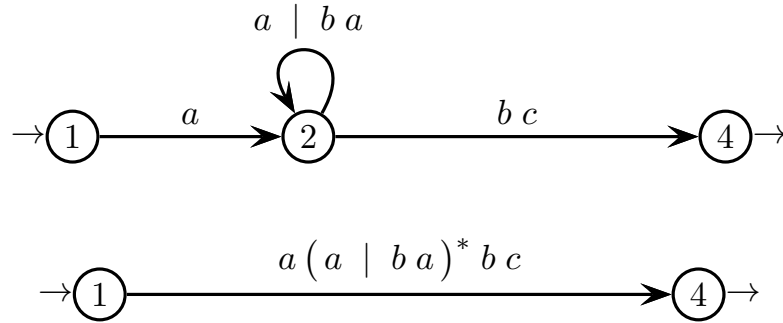
Eliminate node 3:



Therefore we obtain the regular expression R below:

$$R = (a^+ b)^+ c$$

Alternatively, if node 3 were eliminated first and node 2 last, see:



the equivalent formulation below would be obtained:

$$R = a (a \mid b a)^* b c = a ([b] a)^* b c$$

where the square brackets indicate optionality.

- (f) In the strings of the intersection language $L_{\cap}$, differently from the strings of languages $L(R_1)$ and $L(R_2)$, the letters b and c are immediately followed by letters c and d , respectively, therefore it holds $R_{\cap} = a^+ b c d$.

2 Free Grammars and Pushdown Automata 20%

1. Consider the two languages L_1 and L_2 defined below, over the three-letter alphabet $\Sigma = \{a, b, c\}$:

$$L_1 = \{ a^i c b^{j+i} \mid 1 \leq i \leq 3 \wedge j \geq 0 \}$$

$$L_2 = \{ a^i c^k b^{j+i} \mid 1 \leq i \leq 3 \wedge j \geq 0 \wedge k \leq j \}$$

Answer the following questions:

- (a) Write a non-ambiguous BNF grammar G_1 that generates language L_1 .
(b) Draw the syntax tree of the sample valid string below, with grammar G_1 :

$a a c b b b b$

- (c) Is language L_1 regular or context-free? Justify your answer.
(d) Write a BNF grammar G_2 that generates language L_2 (possibly modify grammar G_1). It does not matter that grammar G_2 is ambiguous or not.
(e) (optional) Consider the intersection language $L_3 = L_1 \cap L_2$. Write down language L_3 as a string pattern constrained by a characteristic predicate, that is:

$$L_3 = \{ \text{string model(s)} \mid \text{characteristic predicate} \}$$

Does the sample string below belong to language L_3 ?

$a c b$

Is language L_3 regular or context-free? Justify your answers.

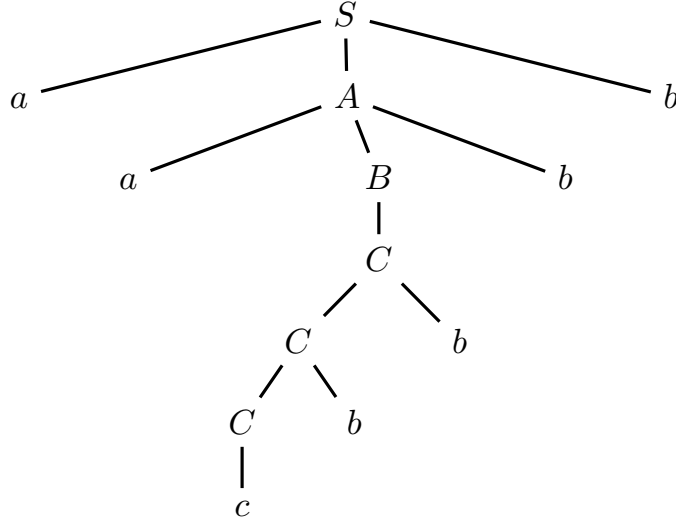
Solution

- (a) Here is a viable non-ambiguous BNF grammar G_1 (axiom S):

$$G_1 \left\{ \begin{array}{l} 1: S \rightarrow a A b \\ 2: A \rightarrow a B b \mid C \\ 3: B \rightarrow a C b \mid C \\ 4: C \rightarrow C b \mid c \end{array} \right.$$

Grammar G_1 generates from one to three nested pairs ab by means of rules 1, 2 and 3. At rule 2 or 3, instead of nesting a pair ab the grammar can move on to rule 4 and insert a sequence (possibly empty) made of letters b and terminated by letter c . Thus the grammar generates exactly language L_1 . The grammar consists of four non-ambiguous rules, their order of application is fixed, i.e., $1 < 2 < 3 < 4$, and only rule 4 is repeatable, thus the grammar is not ambiguous.

- (b) Here is the syntax tree of the sample valid string $aacbbbb$:



Since grammar G_1 is non-ambiguous, the string has one syntax tree.

- (c) Language L_1 is regular. In fact, even though language L_1 includes the string pattern $a^i b^i$, which is well-known for not being regular, the finite constraint $i \leq 3$ allows the language to be recognized by a FSA. In fact, L_1 is generated by the following regular expression R_1

$$R_1 = acbb^* \mid a^2cb^2b^* \mid a^3cb^3b^*$$

- (d) Grammar G_2 , still BNF, has the same rules 1, 2 and 3 (and axiom S) as has grammar G_1 , plus the modified rule 4 below, which is ambiguous:

$$4: C \rightarrow c C b \mid C b \mid \varepsilon$$

or, if a non-ambiguous formulation is preferred, plus the two rules 4 (modified) and 5 (new) below:

$$\left\{ \begin{array}{l} 4: C \rightarrow c C b \mid D \\ 5: D \rightarrow D b \mid \varepsilon \end{array} \right.$$

which impose a derivation order and eliminate ambiguity.

- (e) Languages L_1 and L_2 have the same basic structure: the ordering of the letters is the same and the exponents are partly similar. Consequently, elementary numerical inequations are sufficient to find a description of the intersection language L_3 , based on a characteristic predicate, as follows:

$$L_3 = \{ a^i c b^{i+j} \mid 1 \leq i \leq 3 \wedge j \geq 1 \}$$

We find that language L_3 must have at least one more letter b than is the number of letters a , which was not the case for languages L_1 or L_2 . As a result, the string $a c b$ does not belong to L_3 . Furthermore, language L_3 is regular, like L_1 , and a possible regular expression R_3 for L_3 is as follows:

$$R_3 = a c b b^+ \mid a^2 c b^2 b^+ \mid a^3 c b^3 b^+$$

or equivalently:

$$R_3 = a (c \mid a (c \mid a c b) b) b b^+$$

Indeed both languages L_1 and L_3 share the same structure.

2. Consider the fragment of a language that describes multivariate (with one or more different variables) polynomials. This fragment features the structure as shown in the two sample polynomials below:

$$3 + 5 x^2 + 2 x^3 y$$

$$2 x + 3 y^2 (8 + 4 x z)^2$$

The language has the following characteristics:

- a polynomial is a sum “+” of monomials
- a monomial is the product (the multiplication sign is constantly understood) of one coefficient (a positive integer), mandatory, and of zero, one or more variables (identifiers), each of them possibly with a positive integer exponent, marked by the operator “^”; in place of any variable, there may be a subexpression consisting of a polynomial enclosed in parentheses
- addition is left-associative and multiplication is right-associative
- the precedence order from high to low is first exponentiation, then multiplication and finally addition
- for simplicity, positive integers and variables are schematized by the terminals `num` and `var`, respectively; neither terminal has to be further expanded

Answer the following questions:

- Write an EBNF non-ambiguous grammar that generates the language described before (the axiom must generate one polynomial).
- Draw the syntax tree of the sample polynomial shown below:

$$5 x + 2 x y^3$$

- (optional) Suppose the subtraction “−” of monomials is also possible, like:

$$3 + 5 x^2 - 2 x y^3$$

Subtraction is not associative, i.e., the grammar and the tree should not enforce either type (left or right) of associativity. Subtraction takes precedence over addition and gives precedence to multiplication. Accordingly modify the previous grammar, still non-ambiguous (you may show only the modified or new rules).

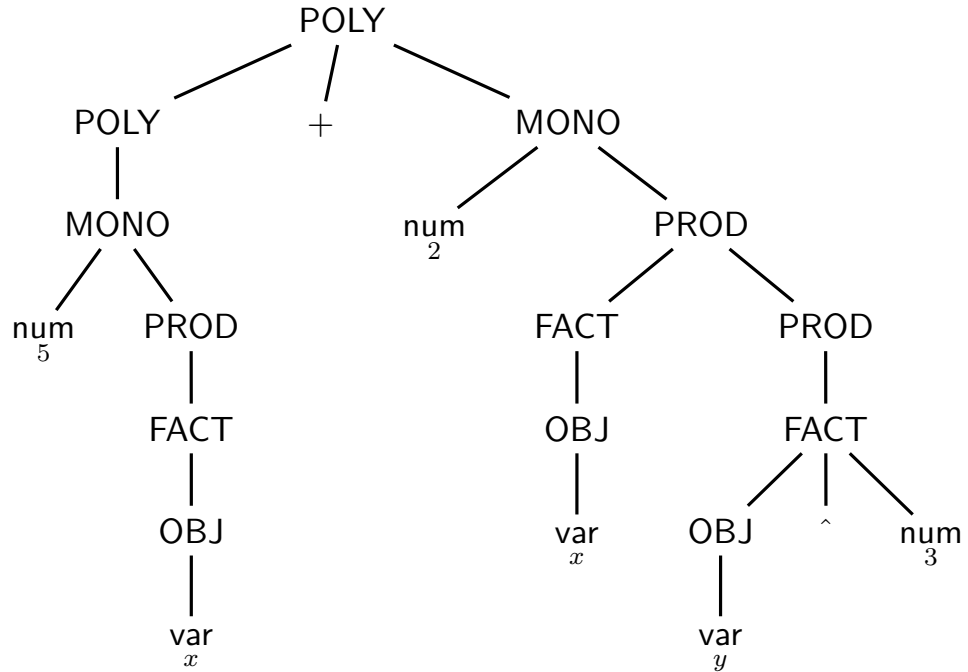
Solution

- (a) Here is a viable non-ambiguous grammar (axiom POLY):

$$\left\{ \begin{array}{l} \langle \text{POLY} \rangle \rightarrow \langle \text{POLY} \rangle \text{ "+" } \langle \text{MONO} \rangle \mid \langle \text{MONO} \rangle \quad // \text{ left-recurs.} \\ \hline \langle \text{MONO} \rangle \rightarrow \text{num } \langle \text{PROD} \rangle \\ \langle \text{PROD} \rangle \rightarrow \langle \text{FACT} \rangle \langle \text{PROD} \rangle \mid \langle \text{FACT} \rangle \quad // \text{ right-recurs.} \\ \hline \langle \text{FACT} \rangle \rightarrow \langle \text{OBJ} \rangle [\text{"^"} \text{ num }] \\ \langle \text{OBJ} \rangle \rightarrow \text{var} \mid \text{"(" } \langle \text{POLY} \rangle \text{ ")" } \end{array} \right.$$

where the square brackets indicate optionality.

- (b) Here is the syntax tree of the sample polynomial $5x + 2xy^3$. The possible expansions of the terminal leaf nodes `num` and `var` are shown below each node:



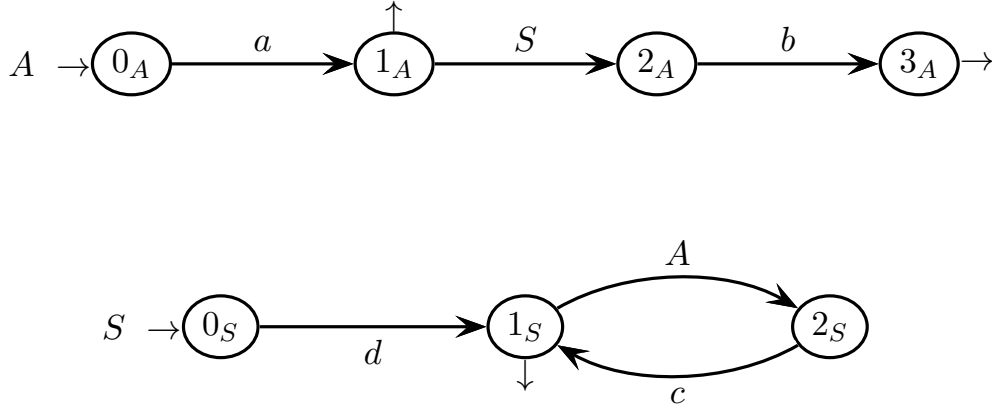
- (c) Here is the new grammar, where the modified parts of the rules and the new rules are coloured in **red** (thus we could just show the two top rules):

$$\left\{ \begin{array}{l} \langle \text{POLY} \rangle \rightarrow \langle \text{POLY} \rangle \text{ "+" } \langle \text{SUB} \rangle \mid \langle \text{SUB} \rangle \quad // \text{ left-recursive} \\ \langle \text{SUB} \rangle \rightarrow \langle \text{MONO} \rangle (\text{"-"} \langle \text{MONO} \rangle)^* \quad // \text{ not recursive} \\ \hline \langle \text{MONO} \rangle \rightarrow \text{num } \langle \text{PROD} \rangle \\ \langle \text{PROD} \rangle \rightarrow \langle \text{FACT} \rangle \langle \text{PROD} \rangle \mid \langle \text{FACT} \rangle \quad // \text{ right-recursive} \\ \hline \langle \text{FACT} \rangle \rightarrow \langle \text{OBJ} \rangle [\text{"^"} \text{ num }] \\ \langle \text{OBJ} \rangle \rightarrow \text{var} \mid \text{"(" } \langle \text{POLY} \rangle \text{ ")" } \end{array} \right.$$

Note that the rule of nonterminal `SUB` does not enforce any specific type of associativity for subtraction, as the rule is not recursive, but iterative. The precedence of subtraction is intermediate between addition and multiplication.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar G below, represented by a machine net with the two-symbol nonterminal alphabet $V = \{S, A\}$ (axiom S) and the four-letter terminal alphabet $\Sigma = \{a, b, c, d\}$:



Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Draw the syntax trees for the two sample valid strings below:

$$d a d b c \in L(G) \qquad d a c a d b c \in L(G)$$

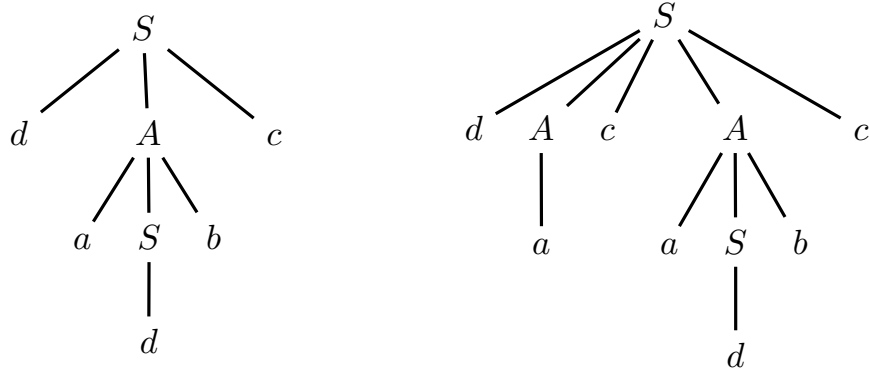
- (b) Draw the complete pilot of grammar G and argue that grammar G is ELR(1).
- (c) Simulate the ELR(1) parsing process of the sample valid string below:

$$d a d b c \in L(G)$$

- (d) Write all the guide sets on the call arcs and exit arrows of the net of grammar G , and argue that grammar G is ELL(1).
 - (e) (optional) Write the recursive-descent syntactic procedure for nonterminal S .
-

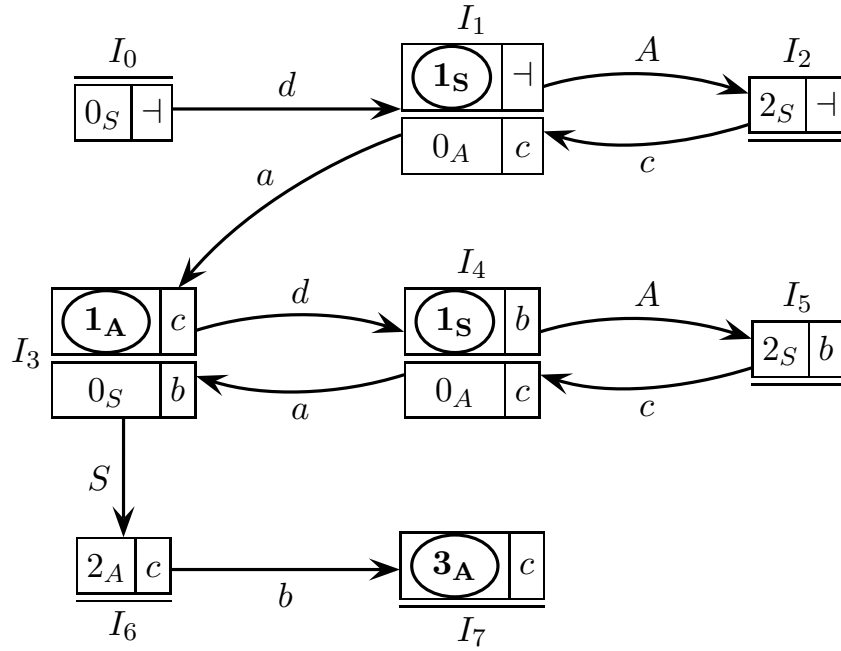
Solution

- (a) Here are the syntax trees for the two sample valid strings $dadbcb$ and $dacadbcb$:



Grammar G is not ambiguous (since in the text it is said to be ELR and in fact it is so, see point (b)), thus there is only one syntax tree for each valid string.

- (b) Here is the complete pilot of grammar G , with eight m-states, and its analysis:

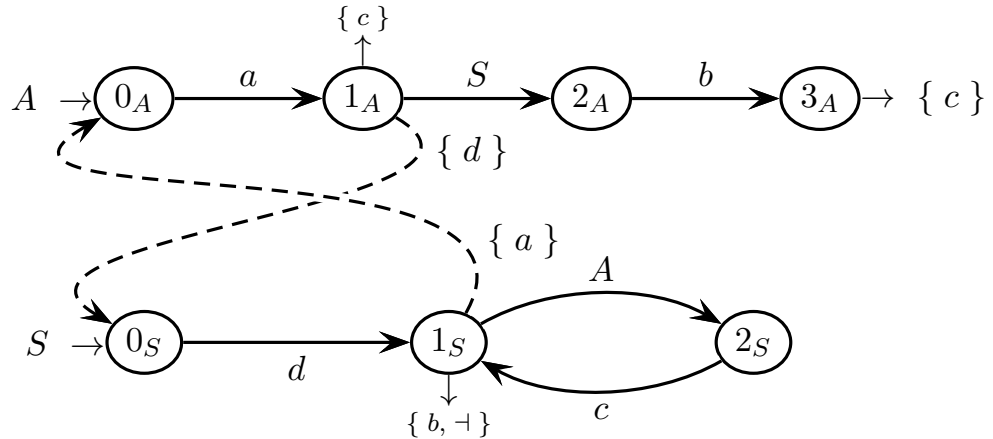


There are neither SR conflicts, as the look-ahead symbols on the final items (in the m-states I_1 , I_3 , I_4 and I_7) are disjoint from those on the outgoing terminal shift transitions, nor RR conflicts, as no m-state with final items (see the list of m-states before) contains more than one final item, nor multiple transitions, as the base of every m-state contains at most one item (the pilot has the STP), thus there are no convergence conflicts either. Consequently grammar G is ELR (1).

- (c) Here is the simulation of the ELR (1) parsing process of the valid sample string $dadbcb \in L(G)$:

<i>stack of m-states and grammar symbols</i>	<i>input left to scan</i>	<i>move executed</i>
I_0	$dadbcb \dashv$	initial configuration
$I_0 d I_1$	$adbcb \dashv$	terminal shift: $I_0 \xrightarrow{d} I_1$
$I_0 d I_1 a I_3$	$dbcb \dashv$	terminal shift: $I_1 \xrightarrow{a} I_3$
$I_0 d I_1 a I_3 d I_4$	$bcb \dashv$	terminal shift: $I_3 \xrightarrow{d} I_4$
$I_0 d I_1 a I_3$	$bcb \dashv$	reduction: $d \rightsquigarrow S$
$I_0 d I_1 a I_3 S I_6$	$bcb \dashv$	nonterminal shift: $I_3 \xrightarrow{S} I_6$
$I_0 d I_1 a I_3 S I_6 b I_7$	$c \dashv$	terminal shift: $I_6 \xrightarrow{b} I_7$
$I_0 d I_1$	$c \dashv$	reduction: $aSb \rightsquigarrow A$
$I_0 d I_1 A I_2$	$c \dashv$	nonterminal shift: $I_1 \xrightarrow{A} I_2$
$I_0 d I_1 A I_2 c I_1$	$\dashv$	terminal shift: $I_2 \xrightarrow{c} I_1$
I_0	$\dashv$	reduction: $dAc \rightsquigarrow S$
initial stack	empty input	last move reduction to axiom
acceptance condition verified – see also the tree at point (a)		

- (d) Here are all the guide sets on the call arcs and exit arrows of the net of grammar G , and the analysis of the PCFG:



The arcs from the nodes with bifurcations, i.e., nodes 1_S and 1_A , have disjoint guide sets, thus grammar G is ELL (1). This was expected since grammar G is ELR, it is not left-recursive and its pilot has the STP.

- (e) Here is the recursive-descent procedure for nonterminal S , in the straightforward version that faithfully maps the graph of machine S :

```

// straightforward version - faithfully maps the
graph
procedure  $S$                                 // state mapping
if  $cc \in \{d\}$  then                          // state  $0_S$ 
     $cc := next$                                 // goto  $1_S$ 
    LOOP: if  $cc \in \{a\}$  then                // state  $1_S$ 
        call  $A$                                 // goto  $2_S$ 
        if  $cc \in \{c\}$  then                // state  $2_S$ 
             $cc := next$                         // goto  $1_S$ 
            goto LOOP                        // back to  $1_S$ 
        else error                            // state  $2_S$ 
    else if  $cc \in \{b, \neg\}$  then return      // exit  $1_S$ 
    else error                                // state  $1_S$ 
else error                                    // state  $0_S$ 
end procedure

```

With some optimization, i.e., grouping the error cases at the end and making the loop explicit, the procedure becomes more compact:

<pre> // error cases at the end procedure S if $cc \in \{d\}$ then $cc := next$ LOOP: if $cc \in \{a\}$ then call A if $cc \in \{c\}$ then $cc := next$ goto LOOP else if $cc \in \{b, \neg\}$ then return error // states 0_S, 1_S and 2_S end procedure </pre>	<pre> // with explicit loop procedure S if $cc \in \{d\}$ then $cc := next$ while $cc \in \{a\}$ do call A if $cc \in \{c\}$ then $cc := next$ else error // state 2_S if $cc \in \{b, \neg\}$ then return error // states 0_S and 1_S end procedure </pre>
---	--

In the version with explicit “while” loop (top right), we cannot move the “**else error**” clause from inside the loop to the end of the procedure with the others. In fact, suppose we have moved the clause: then in the case in which the “**if**” conditional inside the loop failed, i.e., $cc \neq c$, because by chance upon returning from procedure A we have $cc = a$, the loop would repeat; yet such a behaviour is wrong. The other optimized version (top left) does not behave so, since the “goto” statement is executed only if the condition $cc \in \{c\}$ agrees.

In reality the **clause** can even be deleted, provided that procedure A checks on return whether the input contains letter c (see the exit look-ahead symbol of machine A), and that it halts on error if the input does not contain it. Then also the **conditional** can be deleted, just keeping the shift statement $cc := next$. Of course, the deletion presupposes that procedure A is actually coded as said. This version with optimized checks looks like as follows:

```
// with optimized checks
procedure  $S$ 
if  $cc \in \{d\}$  then
   $cc := next$ 
  while  $cc \in \{a\}$  do
    call  $A$ 
     $cc := next$ 
  if  $cc \in \{b, \neg\}$  then return
error
end procedure
```

As mentioned however, its correctness also depends on whether the rest of the analyzer, i.e., procedure A , performs the appropriate check and halt moves.

4 Language Translation and Semantic Analysis 20%

1. Consider the following source grammar G , over the two-symbol nonterminal alphabet $V = \{S, L\}$ (axiom S) and the three-letter terminal alphabet $\Sigma = \{a, "[", "]" \}$. Grammar G generates nested lists delimited by square brackets:

$$G \left\{ \begin{array}{l} S \rightarrow "[L "]" \\ L \rightarrow "[L "]" L \\ L \rightarrow "[L "]" \\ L \rightarrow a L \\ L \rightarrow a \end{array} \right.$$

Answer the following questions:

- (a) For the translation function τ_1 that flattens the nested lists, for instance:

$$\tau_1([a[a a][[a]]) = [a a a a]$$

write a BNF translation grammar or scheme G_1^τ (with source grammar G).

- (b) Draw the translation tree for the sample translation below:

$$\tau_1([a[a a][[a]]) = [a a a a]$$

- (c) For the translation function τ_2 that removes all the square brackets and terminates each (sub)list with a single dot ".", for instance:

$$\tau_2([a[a a][[a]]) = a a a . a \dots$$

write a BNF translation grammar or scheme G_2^τ (with source grammar G).

- (d) Draw the translation tree for the sample translation below:

$$\tau_2([a[a a][[a]]) = a a a . a \dots$$

- (e) Are the two translations τ_1 and τ_2 invertible? Why?

Solution

- (a) Here is a translation grammar G_1^T (with source G) that flattens the nested lists:

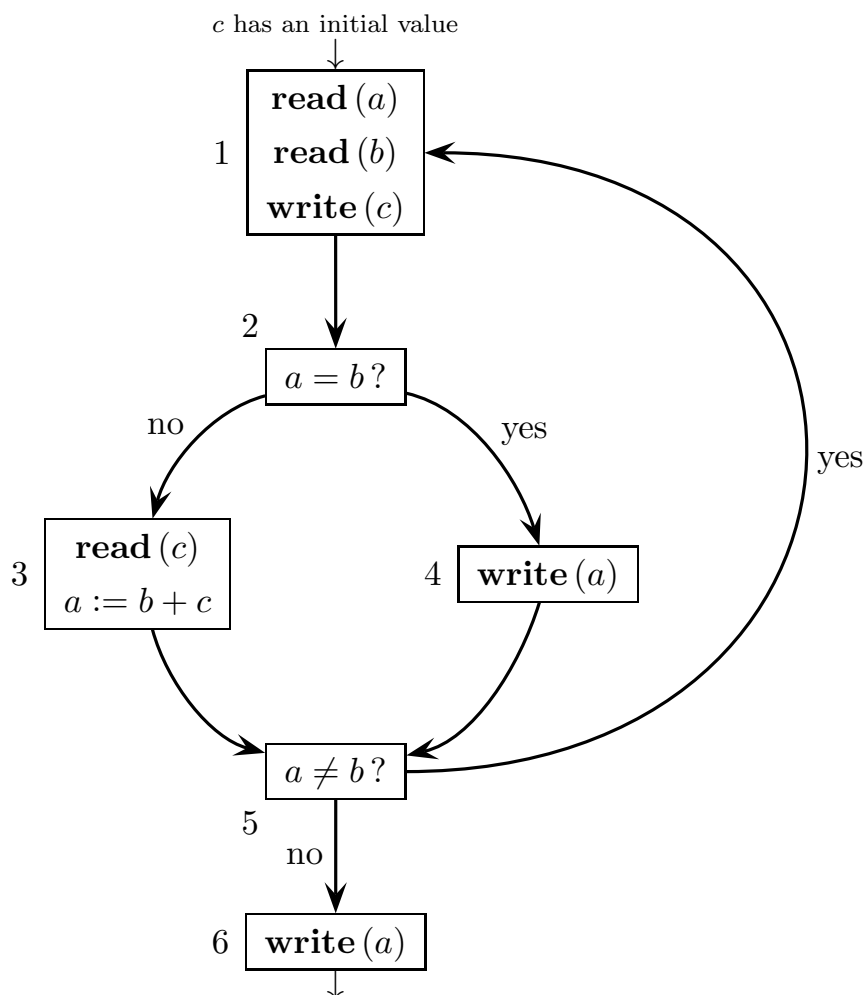
$$G_1^T \left\{ \begin{array}{l} S \rightarrow \frac{[""]{}}{[""]} L \frac{[""]{}}{[""]} \\ L \rightarrow \frac{[""]{}}{\varepsilon} L \frac{[""]{}}{\varepsilon} L \\ L \rightarrow \frac{[""]{}}{\varepsilon} L \frac{[""]{}}{\varepsilon} \\ L \rightarrow \frac{a}{a} L \\ L \rightarrow \frac{a}{a} \end{array} \right.$$

- (b) Translation tree for $\tau_1([a[aa][[a]]) = "[aaaa]":$ see above right.
- (c) Here is a translation grammar G_2^T (with source G) that removes the brackets and terminates each (sub)list with a dot “.”:

$$G_2^T \left\{ \begin{array}{l} S \rightarrow \frac{["}{\varepsilon} L \frac{"]}{\cdot} \\ L \rightarrow \frac{["}{\varepsilon} L \frac{"]}{\cdot} L \\ L \rightarrow \frac{["}{\varepsilon} L \frac{"]}{\cdot} \\ L \rightarrow \frac{a}{a} L \\ L \rightarrow \frac{a}{a} \end{array} \right.$$

- (d) Translation tree for $\tau_2([a[a a][[a]]) = "a a a . a \dots"$: see above right.
- (e) Here is the analysis of the invertibility of the two translation functions:
- τ_1 is not invertible, as the source bracketing is completely lost (except for the outermost bracket pair) and cannot be uniquely reconstructed
 - τ_2 is not invertible, as the dot allows to reconstruct the source closed bracket of each (sub)list, but not the open one, thus for instance the list $[a[a]]$ cannot be distinguished from $[[a a]]$, since translation τ_2 maps both to $"a a ."$

2. Consider the control flow-graph (CFG) of the program below, with three numerical variables a , b and c (variable c has an initial value passed as input argument), and six nodes numbered 1, $\dots$, 6:



Answer the following questions (use the tables / trees / spaces on the next pages):

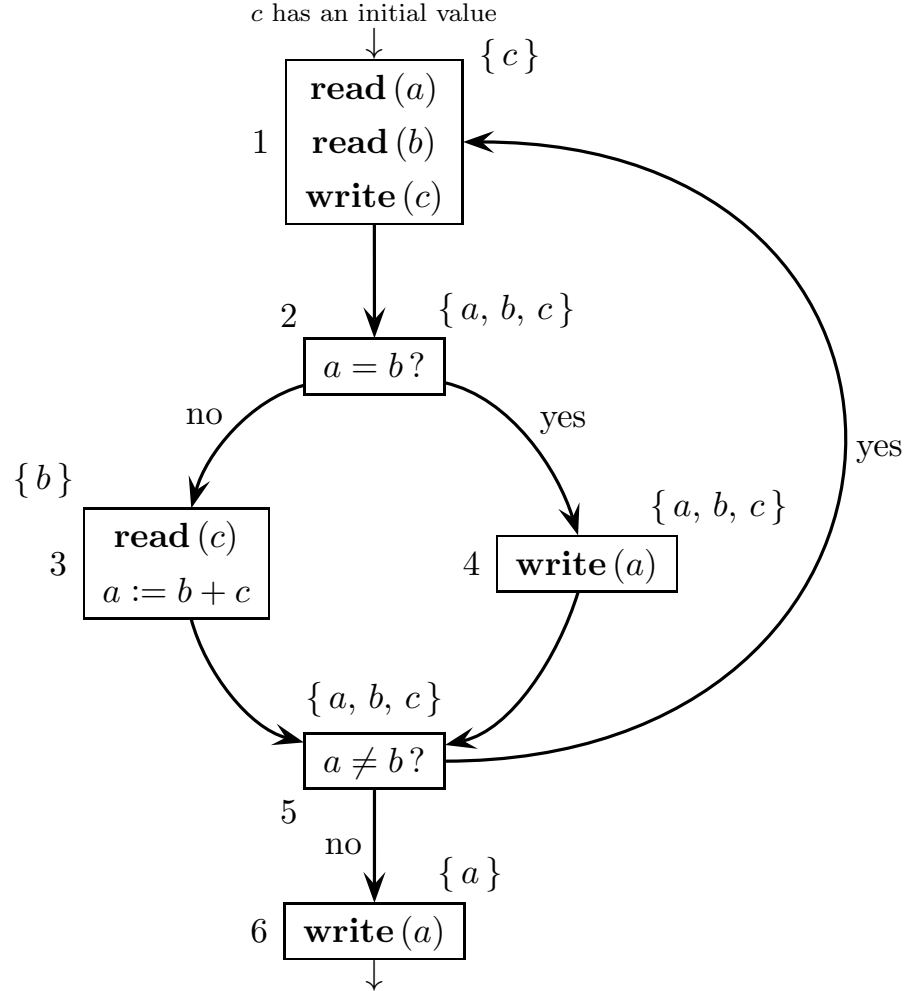
- Write a regular expression R that generates the language of the program execution traces, over the alphabet $\{1, \dots, 6\}$, disregarding the program semantics.
- Informally find the live variables and decorate each node of the graph with the variables that are live at the input of the node (see the next pages).
- Write the liveness flow equations for the node 3 and then verify that the live variables informally found at point (b) satisfy such equations.
- How many definitions does the variable a have? Decorate each node of the graph with the definitions of a that reach the output of the node (see the next pages).
- (optional) Now consider the semantics of the program. Can you change the regular expression R to a more restrictive one R' , such that $L(R')$ is strictly contained in $L(R)$?

Solution

(a) Here is a viable regular expression R for the execution traces of the program:

$$R = (1\ 2\ 3\ 5 \mid 1\ 2\ 4\ 5)^+ 6 = (1\ 2\ (3 \mid 4)\ 5)^+ 6$$

(b) Here are the live variables (on the node input):

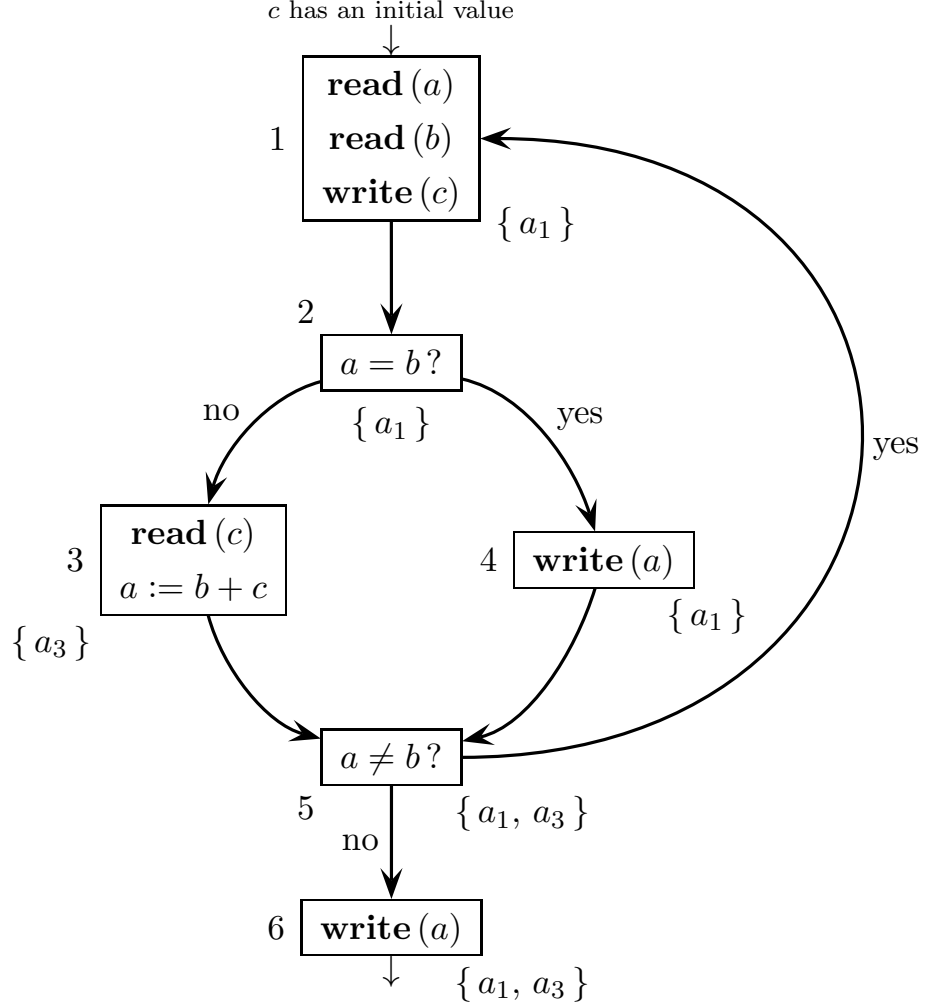


(c) At node 3, the variables a and c are defined, and the variable b is used. The liveness equations of node 3 are:

$$in(3) = \{b\} \cup (out(3) - \{a, c\}) \quad out(3) = in(5)$$

Since $in(5) = \{a, b, c\}$, then $in(3) = \{b\} \cup (\{a, b, c\} - \{a, c\}) = \{b\}$, thus the informal solution (on node 3) is immediately verified.

- (d) There are two definitions for variable a , namely a_1 and a_3 . Here are the reaching definitions (on the node output):



- (e) According to the program semantics, we can distinguish three cases:

- if at node 2 it does not hold $a = b$ and at node 3 the read assigns to c a value $\neq 0$, so that a remains different from b , then the loop is repeated
- if at node 2 it does not hold $a = b$ and at node 3 the read assigns 0 to c , so that a becomes equal to b , then the program exits with sequence 3 5 6
- if at node 2 it holds $a = b$, then the program exits with sequence 4 5 6

Thus in every execution trace node 4 is either skipped or executed only once before exiting. Then we can restrict the execution trace language as follows:

$$R' = (1\ 2\ 3\ 5)^* (1\ 2\ 3\ 5\ 6 \mid 1\ 2\ 4\ 5\ 6) = (1\ 2\ 3\ 5)^* 1\ 2\ (3 \mid 4)\ 5\ 6$$

and it is evident that $L(R') \subset L(R)$, i.e., strict containment.